TER: A BALANCED-TERNARY SUBSTRATE SIMULATOR THAT RUNS LLAMA 3.2 1B FASTER THAN LLAMA.CPP Q8_0 AND REPRODUCES BITNET 2B-4T BIT-EXACT AT 1.58 BITS PER WEIGHT ON CONSUMER GPUS

A PREPRINT

Samuel Cortes
https://naranjositos.tech/
https://github.com/xaskasdf/ter

May 2026

ABSTRACT

We present `ter`, a balanced-ternary CPU simulator with SIMD extension and bytecode kernels (`tk_matmul_b_9t`, `tk_rmsnorm`, `tk_silu`, `tk_softmax`, `tk_rope`). The simulator serves as the research vehicle: modern transformer language models—Llama 3.2 1B (Q8_0, post-training ternarized), BitNet b1.58 2B-4T (i2_s, QAT-trained), TinyStories Llama 2 20M, and brandon-tiny 10M—run end-to-end on the substrate with finite outputs and operation-count parity ($1.002\times$ lane-MACs vs the analytical fp16 baseline).

To enable architectural iteration at sweepable speed, we ported the inner matmul to CUDA. The paper reports two engineering results in this GPU regime: (i) on Llama 3.2 1B, a round-2 engineering pass on the end-to-end forward (device-pointer scales, warp-cooperative packed-trit kernel, `cudaGraph` capture, and rmsnorm/quant fusion) reaches $\mathbf{425}$ tokens/s on RTX 3090, $\mathbf{1.08}\times$ faster than `llama.cpp` Q8_0; (ii) on BitNet b1.58 2B-4T, integration of real `microsoft/BitNet` i2_s weights with sub-RMSNorms, ReLU^2 FFN, NEOX RoPE, fp32 residual stream, and a ternary-only ADD-only matmul kernel reaches 214 tokens/s with $\mathbf{8/8}$ BOS-greedy tokens bit-exact against the `microsoft/BitNet` `llama-cli` reference.

We are explicit about what is and is not ternary about these results. The matmul-fabric speedup of $1.90\times$ over cuBLAS INT8 tensor cores at single-token gen is attributable to packed sub-byte storage (4 trits per byte), not to ternary arithmetic per se: the formal analysis of Cuevas (2026) decomposes $\sim 1.88\times$ as a bandwidth ratio (16 MACs per 17 bytes read versus $1/2$ for INT8), and our own ADD-only experiment shows that eliminating multiplications does not yield throughput on commodity GPU. The genuinely ternary contribution is bit-exact reproduction of the QAT-trained BitNet 2B-4T model, where substrate-data alignment is semantically meaningful. The per-op energy advantage that historically motivated ternary CPUs—and that the simulator was designed to explore—remains unmeasured and requires an FPGA prototype to close. Related LUT-based low-bit work on CPU Wei et al. [2025], FPGA Xie et al. [2025], Qiao et al. [2025], and ASIC Shan et al. [2025], Scherer et al. [2020] precedes ours; we contribute the specific GPU `__dp4a` packed-trit kernel design plus end-to-end engine integration with bit-exact validation.

Keywords balanced ternary computing · LLM inference · CUDA optimization · INT8 tensor cores · substrate-data alignment · BitNet b1.58 · Llama 3.2 1B

1 Introduction

The dominant story of LLM inference acceleration has been quantization into ever-tighter binary formats: fp32 → fp16 → int8 → int4 → binary. Recent work [Wang et al., 2023, Ma et al., 2024] demonstrates that ternary weights

($\{-1, 0, +1\}$) can match the perplexity of fp16 baselines at similar parameter counts when training is quantization-aware. This paper asks the natural follow-up: *if the data is ternary, what does the optimal substrate look like, and can we measure the win on existing hardware?*

We answer this in three parts:

1. Build a balanced-ternary CPU simulator faithful enough to run modern transformer architectures end-to-end (Llama 3.2 1B Q8_0, BitNet b1.58 2B-4T i2_s).
2. Validate three hypotheses about the substrate at the architectural (operation-count) level.
3. Port the matmul fabric to CUDA (as an iteration aid for the simulator, and to measure the wall-clock behavior of packed sub-byte ternary storage on existing silicon, an NVIDIA RTX 3090).

The two engineering results that this paper defends are: (a) end-to-end Llama 3.2 1B inference at **425** tokens/s on RTX 3090 ($28.9\times$ over our v1 baseline, $1.08\times$ faster than `llama.cpp` [Gerganov, 2023] Q8_0); and (b) bit-exact reproduction of `microsoft/BitNet` 2B-4T at 214 tokens/s ($8/8$ BOS-greedy tokens match the reference implementation).

The GPU matmul-fabric measurement underlying (a) is a $1.90\times$ speedup over cuBLAS INT8 tensor cores at single-token gen on the Llama 1B shapes. We attribute this speedup to packed sub-byte storage (4 trits per byte, 1.58 bits per weight) rather than to ternary arithmetic itself, following the formal decomposition of Cuevas Araneda [2026] (§5.2): bandwidth density yields $\sim 1.88\times$, and our own ADD-only kernel that eliminates multiplications *matches* but does *not exceed* the `__dp4a`-based packed kernel. The same advantage would in principle accrue to a generic two-bit packing scheme; we discuss this honestly in §6.3. Per-op energy in custom silicon—the historical motivation for ternary CPUs, and the original target of the substrate simulator—remains unmeasured here and requires an FPGA prototype to close. Related LUT-based low-bit acceleration work [Wei et al., 2025, Xie et al., 2025, Shan et al., 2025, Qiao et al., 2025, Scherer et al., 2020] precedes ours; our contribution is the specific GPU packed-trit kernel design plus the end-to-end engine integration validated bit-exact against a published reference.

The simulator is at `https://github.com/xaskasdf/ter`; all benchmarks are reproducible.

2 Related work

BitNet b1.58 [Ma et al., 2024] demonstrated that ternary weights ($\{-1, 0, +1\}$) match fp16 perplexity when training is quantization-aware. We treat their model class as the canonical target for substrate-data alignment (§4.3).

microsoft/BitNet [Microsoft, 2024] provides the CPU inference framework with custom kernels (TL1 / TL2 backends) specialized to ternary weights. We benchmark our simulator against their published 2B-4T model checkpoint.

llama.cpp [Gerganov, 2023] is the reference CPU/GPU inference engine for GGUF-format models. We use it as the production binary baseline for our wall-clock comparisons.

Setun [Brusentsov, 1959] was the first commercial balanced-ternary computer (Moscow State University, 1959). The information-density argument for radix 3 (closer to e than radix 2) is its primary motivation. Modern follow-ups [Vudadha et al., 2016, Hayes et al., 2018] report 25–40% per-MAC energy savings for ternary CMOS ALU circuits versus equivalent-precision binary.

Llama 3.2 1B [Meta AI, 2024] is our reference mid-scale transformer; we use the Q8_0 quantized GGUF from the official HuggingFace mirror.

Brandon-Tiny 10M [Cortes, 2026a] is the author's prior 10M-parameter instruction-following LLM; used here as a tiny end-to-end smoke target.

LUT-based low-bit acceleration. The principle that low-bit weight multiplications can be replaced by precomputed table lookups has been demonstrated on multiple platforms. **T-MAC** [Wei et al., 2025] achieves up to $6.6\times$ speedup over `llama.cpp` on CPU for 1–4 bit weights via bit-wise table lookup, with $\sim 70\%$ energy reduction. **LUTMUL** [Xie et al., 2025] exceeds the conventional DSP-based roofline on FPGA by exploiting the $100:1$ LUT-to-DSP ratio. **Platinum** [Shan et al., 2025] adds path-adaptable LUT-based acceleration for low-bit matrix multiplication. **TeLLMe** [Qiao et al., 2025] reports an energy-efficient ternary LLM accelerator for edge FPGAs covering both prefill and decode. **CUTIE** [Scherer et al., 2020] demonstrates ternary DNN inference at > 1 PetaOp/s/W in ASIC silicon. None of these works require "ternary arithmetic" as a hardware primitive: they exploit that low-precision weights admit small precomputed tables.

Ternary weight networks. Li et al. [2016] are the historical predecessor, restricting weights to $\{-\alpha, 0, +\alpha\}$ with a learned scale per layer; their work establishes the model-level viability of post-training and QAT ternarization.

Energy models and critique. Horowitz [2014] provides the per-operation pJ baselines (45nm CMOS) for MUL/ADD that we use to reason about silicon-investment tradeoffs. Meiner et al. [2025] argues that reducing multiplication count is more valuable than further reducing bit-width. Cuevas Araneda [2026] gives a formal analysis specifically relevant to this paper: (i) a lower bound of $\rho_3 \geq 3.8\times$ energy savings per MAC for ternary weights with INT8 activations on 45nm silicon (consistent with the empirical $71.4\times$ reported in [Ma et al., 2024] on 7nm); (ii) a proof that the efficiency $\eta(k) = \log_2(k)/C_{\text{MAC}}(k)$ over symmetric weight alphabets $\mathcal{W}_k$ is maximized at $k = 3$; (iii) a formal MAC-to-LUT reduction. The same author also publishes a critical review of an earlier draft of the present work which motivates the honest decomposition we now make in §6.3—most notably the observation that the $1.90\times$ matmul-fabric speedup on commodity GPU is attributable to packed sub-byte storage density rather than to ternary arithmetic semantics. We follow that critique in this revision.

3 Substrate architecture

3.1 Number formats

Format B (production): integer 9-trit payload plus a per-tensor float32 scale. Each element occupies a balanced-ternary representation with range $[-9841, +9841]$ $(= \pm(3^9 - 1)/2)$. Conceptually equivalent to a INT14 value with implicit per-tensor scaling, used for all loaded model weights.

Format A (`tfloat`, comparison): 1 trit sign + 5 trits exponent + N trits mantissa. Default N=9 gives 15 trits/element with dynamic range $\sim 5 \times 10^{-58}$ to $\sim 5 \times 10^{61}$, far exceeding fp16. Smaller N trades precision for trit budget.

3.2 ISA and SIMD extension

The scalar ISA is a balanced-ternary RISC: 27-trit registers (R_0–R_{26}), `TADD`, `TSUB`, `TNEG`, `TABS`, `TCMP`, `TLOAD`/`TSTORE`, branch opcodes (`TBEQ`, `TBNE`, `TBLT`, `TJUMP`, `TCALL`, `TRET`). The SIMD extension adds vector registers (V_0–V_8) holding 27 lanes of 9-trit values each, accumulator registers (A_0–A_2), and the load-bearing `TVMAC` instruction (multiply-accumulate, 27 lanes per instruction). One `TVMAC` performs 27 multiply-accumulate operations of 9-trit by 9-trit values into a 27-trit accumulator.

3.3 Bytecode kernels

The host C++ program orchestrates inference, dispatching hot inner loops as ternary bytecode kernels into the simulator. The kernel catalog covers Llama-arch primitives:

- `tk_matmul_b_9t`: 27-lane integer dot product (one tile of GEMM).
- `tk_rmsnorm` [Zhang and Sennrich, 2019]: RMSNorm via 256-entry rsqrt LUT.
- `tk_softmax`: per-lane exp LUT + reciprocal LUT.
- `tk_silu`: $\text{SiLU}(x) = x \cdot \sigma(x)$ via sigmoid LUT; SwiGLU [Shazeer, 2020] composed host-side.
- `tk_rope`: pure-SIMD paired rotation [Su et al., 2024] over host-prepared inputs.

3.4 End-to-end forward

`forward_token` performs one token through all logical layers, returning logits over the vocabulary. The transformer struct (`BrandonTransformer`, named historically) holds quantized weights, KV caches per layer, and the small per-token state needed for BitNet's sub-norms or brandon's value-residual. Three model loaders share the same forward path:

- `load_brandon_transformer` (block sharing, DenseFormer, register tokens, value residual)
- `load_llama_transformer` (plain Llama-arch with GQA [Ainslie et al., 2023]; supports Llama 3.2 1B, TinyStories Llama 2 20M)
- `load_bitnet_transformer` (Llama-arch + per-block `attn_sub_norm` + `ffn_sub_norm` per [Ma et al., 2024] §3.2)

3.5 Mac AVX2 baseline (host scalar)

End-to-end forward wall-clock on an Intel i9-9880H (no AVX-512):

Model	Wall-clock / forward	Working set
brandon-tiny F16, 10M	∼11 s	∼50 MB
TinyStories Q4_K_M, 20M	∼3 s	∼30 MB
Llama 3.2 1B Q8_0	∼25 s (post AVX2)	4–8 GB
BitNet b1.58 2B-4T i2_s	∼18 min	8–12 GB

The Mac AVX2 baseline establishes feasibility but is not designed for production wall-clock; it is the platform on which architectural hypotheses (§4) were validated.

4 Hypotheses and validation

4.1 H1: Format B preserves Llama 1B quality

`test_llama_smoke` runs Llama 3.2 1B Q8_0 through the substrate at Format B 9-trit. Result: all 128,256 logits are finite with range $[-10.29, 8.83]$, top token id 31845 stable across re-runs.

The op-count metric is the architectural quantity of interest:

$$\text{lane\_MACs} = \text{TVMAC} \cdot 27 \tag{1}$$

$$\text{fp16\_MACs}_{\text{analytical}} = \sum_{\text{layer}} (M \cdot K_q + 2M \cdot K_{kv} + M \cdot K_o + 3M \cdot K_{\text{ffn}}) + M \cdot V \cdot H \tag{2}$$

For Llama 1B at $M{=}1$:

- TVMAC count: 36.1M kernel + 9.75M analytical lm_head = 45.86M
- lane-MACs: 1.238G
- Analytical fp16 baseline: 1.236G
- Ratio: $\mathbf{1.002}\times$ (parity)

Confirms H1: the ternary substrate executes the same matmul work as fp16 at the operation-count level, ± rounding from the K-tile-of-27 padding.

4.2 H2: Format A trit-budget tradeoff

`test_format_a_vs_b_llama` runs Llama 1B four times with weights round-tripped through TFloat encoder/decoder at varying mantissa widths. Each run produces logits; we compare argmax, top-5 overlap, and RMSE versus the Format B 9-trit baseline.

Encoding	trits/elem	argmax	top-logit	RMSE vs B 9-trit
Format B 9-trit (baseline)	9	31845	8.8295	—
Format A 15-trit (1+5+9)	15	15629	8.8334	0.004
Format A 11-trit (1+5+5)	11	31845	8.8291	0.052
Format A 7-trit (1+5+1)	7	19109	11.7529	3.10

Table 1: Format A vs B trit-budget sweep on Llama 1B. The 15-trit and B 9-trit logits are essentially identical (RMSE 0.004); argmax differs by tie-breaking between two near-equal candidates. The 11-trit configuration recovers the B 9-trit argmax with low RMSE, identifying 1+5+5 as a sweet spot. The 7-trit budget breaks both argmax and RMSE.

The 11-trit Format A is paper-relevant: it preserves Llama-quality output at 11 trits/elem versus the 9-trit Format B baseline, but distributes those trits across {sign, exponent, mantissa} rather than as a single fixed-point integer. This trades a 22% trit-count increase for substantially wider dynamic range.

4.3 H3: Substrate-data alignment for BitNet

Analytical: when weights are exactly $\{-1, 0, +1\}$, $y = \sum_k x_k w_k$ collapses to a sign-flipped sum:

$$y = \sum_{k:w_k=+1} x_k - \sum_{k:w_k=-1} x_k \tag{3}$$

which is pure addition/subtraction. Zero TVMAC operations are required; only TVADD/TVSUB (and skips for $w_k = 0$).

test_bitnet_post_quant verifies this on TinyStories layer matmuls: with weights re-quantized to ternary via the BitNet absmean recipe, the TVMAC count for layer matmuls drops from 121,856 to 0, with 25% of weights becoming exact zeros (skipped entirely).

Real GGUF: test_bitnet_gguf_load loads microsoft/bitnet-b1.58-2B-4T-gguf and dequantizes via dequant_i2_s (2-bit packed weights, 4 trits/byte, mapping $\{0 \to 0, 1 \to +1, 2 \to -1\}$). Verification: 100% ternary purity on every weight tensor.

End-to-end forward: test_bitnet_forward runs the full BitNet 2B-4T forward with attn_sub_norm + ffn_sub_norm (the per-tensor scale absorption mechanism per [Ma et al., 2024] §3.2). Result: 18 min/forward AVX2, all 128,256 logits finite, range $[-10.51, 25.54]$.

Counter-experiment: applying the BitNet quantization post-training to Llama 1B Q8_0 (which was *not* trained for ternary) **breaks quality**:

Encoding	argmax	top-5 (truncated)	logit RMSE
Format B 9-trit	31845	[31845, 15629, 4851, ...]	—
BitNet ternarization	27660	[27660, 59829, 64216, ...]	2.39

Table 2: Post-training BitNet quantization on a Q8_0-trained Llama 1B. Argmax flips, 0/5 top-5 overlap, RMSE 2.39 in logit space (range ~ 19).

The substrate is a substrate, not alchemy: it preserves whatever quantization the model was trained for. H3 holds for QAT-trained ternary models; post-training ternarization on non-QAT models is not free.

5 CUDA acceleration: from sim to silicon

The Mac AVX2 simulator's 25.6 s/forward Llama 1B was prohibitive for architectural iteration. Porting the matmul fabric to CUDA [NVIDIA, 2024b,a] brought single-kernel time below 1 ms, enabling sweeps and quality experiments at iteration speed. The more interesting use turned out to be: *exploit the speed to test ternary-specific kernel optimizations and discover whether the architectural memory win translates to wall-clock supremacy on existing binary silicon.*

5.1 Iteration history

We progressively optimized the packed-trit matmul kernel against a cuBLAS INT8 tensor core baseline. Each variant attacked a specific bottleneck:

5.2 The win mechanism

The v11 / v13 kernels with 4 output columns per warp exploit a structural property of the packed-trit storage layout. Each byte holds 4 trits for 4 contiguous output columns sharing the same byte_col. Per K-iteration:

- One byte read from W (8 bits = 4 trits).
- Four trit decodes (bit shifts and either branches or table lookups; we found the comparison-based decode to compile better).
- Four __dp4a calls (each performs 4 int8 MACs).
- Net: **1 byte memory access → 16 useful MACs** = 16:1 information density per byte memory access, versus INT8's 1:1.

Combined with the column-major byte layout (4 contiguous K-bytes = 1 uint32 load) and warp-cooperative K-reduction (32 threads share the K dimension and warp-shuffle their partial sums to lane 0), the packed-trit kernel

Variant	Strategy	ms / forward	GMAC/s
v4 wide	scalar K-loop, 1 col/thread, big block, shared X	384	3.2
v6 dp4a	+ `__dp4a` SIMD (4 MACs / instruction)	172	7.2
v7 colmaj	+ W in column-major byte layout (uint32 contiguous)	99	12.6
v8 warp	warp-cooperative K-reduction (1 warp = 1 col)	29	42
v10 unroll	+ 16-K-element loop body (4 dp4a per body)	14	88
v11 warp4	**4 cols per warp** (1 byte = 4 outputs)	7.65	162
v13 deep	v11 + branchless decode + deep unroll	11.5	108
best dispatch	per-shape kernel selection (v11 or v13)	**6.67**	**186**
cuBLAS INT8 TC	reference (uses third-gen tensor cores)	12.67	98

Table 3: Llama 1B forward (matmul fabric only, $M{=}1$) on NVIDIA RTX 3090. Each line builds on the previous with an additional optimization. Speedup trajectory from naive packed (v4) to best dispatch is $57\times$. The final kernel beats the cuBLAS INT8 tensor core reference by $1.90\times$.

achieves higher throughput than the cuBLAS INT8 tensor-core reference on most Llama 1B shapes at single-token gen. We attribute the gain primarily to memory-bandwidth density (each weight byte delivers 16 MACs of useful work) and discuss the limits of this attribution in §6.3.

5.3 Per-shape results

Shape	$K \times N$	Best packed (GMAC/s)	INT8 TC (GMAC/s)	Ratio
`ffn_down`	8192×2048	303	80	$\mathbf{3.79}\times$ faster
`ffn_gate`	2048×8192	413	131	$\mathbf{3.15}\times$ faster
`lm_head`	2048×128256	374	156	$\mathbf{2.40}\times$ faster
`attn_o`	2048×2048	73	34	$\mathbf{2.13}\times$ faster
`ffn_up`	2048×8192	332	227	$1.46\times$ faster
`attn_v`	2048×512	19	18	$1.06\times$ faster
`attn_q`	2048×2048	75	86	$0.88\times$
`attn_k`	2048×512	19	25	$0.76\times$

Table 4: Per-shape comparison of best packed kernel versus cuBLAS INT8 tensor cores. Six of eight shapes beat the tensor core reference; four of those by 2–4×. The two losses are at the smallest projections ($N{=}512$) where insufficient parallelism limits the warp-cooperative schedule.

5.4 Memory footprint

Layout	MB / Llama 1B	bytes/elem	vs fp16
packed (1.58 b/elem)	**295**	**0.20**	$\mathbf{8.0}\times$ less
INT8	1179	1.0	$2.0\times$ less
Q8_0 (with scales) [Gerganov, 2024]	1326	~ 1.13	$1.78\times$ less
fp16	2357	2.0	—

Table 5: Weight memory footprint per Llama 1B forward (matmul fabric only). Ternary packed at 1.58 bits/element gives $5\times$ less memory than the production Q8_0 baseline and $8\times$ less than fp16.

The end-to-end VRAM for our packed forward implementation is 796 MB (versus 1180 MB for the INT8 baseline, a 33% savings); this can be reduced further to ∼360 MB if the `token_embd` table is also packed (currently kept at fp16 for the embedding lookup, dominating at 525 MB).

5.5 Diminishing returns

After the v11 / v13 generation, additional variants (v12 branchless decode, v13 deep loop unrolling beyond what the compiler already does) showed worse or marginal results. The compiler optimization of comparison-based decodes was already near-optimal; deeper unrolling helped only on large-N shapes; small-N shapes (`attn_q`, `attn_k`) are limited

by parallelism rather than per-thread overhead. Best-per-shape dispatch (selecting v11 or v13 based on N) gave the final 6.67 ms / 186 GMAC/s. We stopped optimizing here: the win is consistent and well-mechanized.

6 End-to-end inference: from 27× behind to 1.08× ahead

A first end-to-end pass (§5.7 of the matmul-fabric work) ran the full Llama 1B forward pipeline (RMSNorm, RoPE, attention, FFN, lm_head, sampling) in CUDA with INT8 tensor core matmuls and delivered 14.7 tokens/s. Against the recalibrated production reference (`llama.cpp` Q8_0 on the same RTX 3090 with a clean GPU: **395** tokens/s, May 2026), this is 27× behind. The previous draft of this paper accepted that gap as "engineering, not architectural". Round 2 *measured* the engineering and closed the gap.

Four cumulative fixes on `ter_cuda_forward_packed.cu`, each verified by clean-GPU bench:

Table 6: Round-2 end-to-end optimization on Llama 3.2 1B (RTX 3090).

Stage	t/s	ms/token	vs `llama.cpp` Q8_0
v1 baseline (`mm_packed_v4` + 65 D2H syncs/token)	14.7	68.2	27× behind
+ device-pointer scale (eliminates D2H syncs)	28.9	34.6	14× behind
+ v11 warp-coop kernel (replaces v4)	200.6	4.99	1.97× behind
+ `cudaGraph` capture (eliminates per-token CPU pacing)	412	2.43	**1.04**× *ahead*
+ rmsnorm/quant + silu/quant fusion (49 launches/token saved)	**425**	**2.35**	**1.08**× *ahead*

Fix 1 (device-pointer scale): the int8 quantization scale was being read back to host via `cudaMemcpy(DeviceToHost)` after every `quant_int8_k` kernel—4 times per layer plus once for `lm_head` = 65 stalls per token. Each stall forces the entire CUDA stream to synchronize. Passing `s.scale_buf` as a `const float*` device pointer to the matmul kernel (which reads the scale into shared memory inside the kernel) and converting K/V cache copies to `cudaMemcpyAsync` doubled throughput.

Fix 2 (kernel upgrade v4 → v11): the end-to-end forward was using the older `mm_packed_v4` kernel (K-tiled shared memory, scalar inner loop) rather than the v11 warp-cooperative kernel that won the matmul-fabric benchmark. Swapping in v11 (col-major W layout, 4 columns per warp, `__shfl_xor_sync` K-reduction, `__dp4a` SIMD inner loop) gave another 6.94× speedup.

Fix 3 (`cudaGraph` capture): eliminated CPU-side per-token state by moving `pos` and `token_id` into device-resident `int*` buffers, plus new helper kernels (`token_embed_lookup_k`, `kv_copy_k`, `inc_pos_k`) and routing `argmax_k` output directly to `s.token_id_dev` to chain into the next forward without host involvement. With all state on-device, the entire forward (340+ kernel launches) is captured once into a `cudaGraph` via `cudaStreamBeginCapture/EndCapture` and replayed per token. `cudaGraph` eliminates not just the ∼3–5 μs CUDA launch latency per kernel but also CPU pacing gaps that prevent the GPU from issuing kernels back-to-back.

Fix 4 (kernel fusion): a single `rmsnorm_quant_fp16_in_k` kernel fuses RMSNorm + int8 quantization into one block-resident pass (reduce sum-of-squares → normalize → reduce amax → quantize). A complementary `silu_mul_quant_k` fuses SiLU + multiply + quantize for the FFN intermediate. Per layer this saves 3 launches; total 49 fewer kernel launches per token on Llama 1B (16 layers).

The final result—**425** tokens/s, **1.08**× faster than `llama.cpp` Q8_0—demonstrates that the packed-trit matmul fabric translates to end-to-end production inference on existing GPU silicon. The substrate runs Llama 3.2 1B at **1.58** bits per weight on a consumer GPU, with the gen path bypassing tensor cores entirely (the hybrid dispatch recovers INT4 tensor cores for prefill regimes where they saturate; see §6.3 for honest attribution of where the win comes from).

6.1 BitNet 2B-4T integration with bit-exact reference match

The same end-to-end engine, retargeted to BitNet 2B-4T architecture (sub-RMSNorms before Wo and `Wdown`, ReLU^2 FFN, NEOX RoPE, weight-tied fp16 `lm_head`) and loaded with real `microsoft/BitNet` i2_s GGUF weights (channel-interleaved 128-element block decoding, $\{0,1,2,3\} \to \{-1,0,+1\}$ remapping, per-tensor scale extraction), reaches **214** tokens/s and produces $\mathbf{8/8}$ BOS-greedy tokens bit-exact against the `microsoft/BitNet` `llama-cli` reference. Achieving the bit-exact match required a fp32 residual stream (BitNet 2B-4T residuals can grow > 65504 by layer 20+, overflowing fp16) and discovering an fp16 overflow on `relu2(gate)*up` (gate$^2\cdot$ up reaches $\sim 10^5$, exceeding fp16 max). A 4-op fused kernel (`relu2_mul_rmsnorm_quant_k`) collapses relu$^2\cdot$mul + sub-RMSNorm + int8 quantization into a single block-resident pass.

We also benchmarked an ADD-only matmul kernel (`mm_bitnet_addonly`) that replaces `__dp4a` with conditional add/sub/skip, exploiting the architectural property that ternary $\{-1, 0, +1\}$ multiplication is degenerate. TVMAC count is exactly 0 in the matmul fabric. At single-token generation regime ($M = 1$), the ADD-only kernel matches the v11 `__dp4a` kernel; at prefill batches ($M \geq 16$), tensor cores win and INT4 TC (via `wmma::s4` fragments) delivers an additional $1.98\times$ over cuBLAS INT8 TC. A hybrid dispatcher (per-shape best-of-toolkit: packed v11 / INT4 TC / cuBLAS INT8 TC) provides the production matmul-fabric win across regimes.

6.2 Llama 1B kernel-correctness validation (caveat closed)

The previous draft accepted a caveat that the 425 tokens/s Llama 1B headline was on synthetic random ternary weights and that correctness against Llama 3.2 1B golden tokens had not been re-validated post-`cudaGraph`. We close that caveat here.

Three artifacts: (a) a GGUF Q8_0 $\rightarrow$ packed-trit converter (`tools/convert_llama_to_packed.py`) applying the BitNet b1.58 per-tensor recipe ($\text{scale} = \text{mean}(|x|)$; $q = \text{clamp}(\text{round}(x/\text{scale}), -1, +1)$) and emitting v11 col-major packed bytes plus an fp32 per-tensor scale; (b) a NumPy reference forward (`tools/llama_pyfwd.py`) mirroring every CUDA kernel in scalar Python; (c) a `load_model_from_bin` path in `ter_cuda_forward_packed.cu` that loads the converter's output and chains a tied-fp16 `lm_head` matmul against `token_embd` (Llama 3.2 1B uses tied embeddings).

Three-way validation:

1. *Architecture sanity (no ternarization, fp32 dequantized weights):* the reference forward generates $7/8$ tokens identical to `llama-cli` Q8_0 from BOS$=$ 128000. The single divergence is the first token (fp32 vs fp16 close-logit sensitivity at pos 0 with no context). Confirms that RoPE convention (Llama uses NORM consecutive-pair rotation, not NEOX), GQA, RMSNorm, SiLU FFN, and the tied-fp16 `lm_head` are correct in the reference.
2. *NumPy reference vs CUDA forward (with per-tensor ternarization):* both produce the identical sequence **16/16** BOS-greedy tokens bit-exact. Every kernel (fused rmsnorm/quant, packed v11 matmul, NORM RoPE, SiLU+mul, fp16 lm_head, two-pass argmax) matches the scalar reference.
3. *Vs Llama Q8_0 golden tokens:* $0/16$ match. Confirms the H1 prediction (§4) that per-tensor BitNet ternarization on a non-QAT model breaks quality; the model collapses to degenerate token cycling (sequence of token 2349 for ~ 78 steps, then token 11, then token 279).

The 425 tokens/s end-to-end headline is therefore a real substrate throughput measurement on the actual ternarized Llama 1B forward (not synthetic random weights). Recovering Llama-quality outputs would require either a quantization-aware-trained ternary Llama (does not exist publicly), Format A 11-trit ($1 + 5 + 5$) with a higher-precision kernel (H1 showed RMSE 0.05 preserves argmax), or post-quantization fine-tuning to recover quality. These are follow-up tasks; the substrate's throughput claim and kernel correctness are now both defensible.

6.3 What is and isn't ternary about this work

The previous draft of this paper attributed the $1.90\times$ matmul fabric speedup to a "balanced-ternary substrate that beats INT8 tensor cores without using them." A formal critique [Cuevas Araneda, 2026] pointed out that this framing conflates two distinct phenomena. We adopt the critique's decomposition here.

Throughput advantage is bandwidth, not arithmetic. At $M = 1$ on Llama 1B shapes, the packed-trit kernel beats cuBLAS INT8 TC by $1.90\times$ because each byte of weight read into a warp yields 16 MACs (four packed trits times four `__dp4a` calls, each consuming four INT8 activation bytes). The total bytes read per output element is 1 weight byte $+16$ activation bytes $= 17$ bytes for 16 MACs, or ≈ 0.94 MACs per byte versus 0.5 MACs per byte for unpacked INT8. The ratio $0.94/0.5 \approx 1.88\times$ accounts for essentially the entire measured $1.90\times$. The semantics of the packed values being ternary $\{-1, 0, +1\}$ rather than arbitrary INT2 $\{0, 1, 2, 3\}$ is incidental to this throughput claim. A generic two-bit packed INT2 kernel feeding `__dp4a` would in principle obtain the same speedup. We acknowledge this explicitly; the matmul-fabric win is a sub-byte packing result, not a ternary-arithmetic result.

Eliminating multiplications does not translate to GPU throughput. We implemented an ADD-only kernel that replaces `__dp4a` with conditional add/sub/skip, exploiting the architectural property that ternary multiplication is degenerate. TVMAC count is exactly zero in this kernel. At $M = 1$ it *matches* the `__dp4a`-based v11 kernel; it does not exceed it. CUDA core ALUs charge equivalent cycles for INT8 MUL and ADD; warp divergence cost on the conditional branch cancels the savings. The energy advantage of the ternary TVMAC-free regime would be observable on custom silicon [Horowitz, 2014, Scherer et al., 2020], not on commodity GPU. Our own measurement is consistent

with the formal result [Cuevas Araneda, 2026, Prop. 2]: $\rho_3 \geq 3.8\times$ on 45nm is an energy bound, not a throughput bound on binary silicon.

Substrate-data alignment is genuine only for QAT models. The H3 claim that ternary substrate aligns with ternary data holds operationally for BitNet b1.58 2B-4T: the model was QAT-trained for ternary weights, and we reproduce $8/8$ BOS-greedy reference tokens bit-exact. For Llama 3.2 1B, post-training ternarization breaks quality (H1, also §6.2): $0/16$ tokens match the Q8_0 golden reference, the model collapses to a degenerate token cycle. The 425 tokens/s headline on Llama 1B is a measurement of substrate throughput on the actual ternarized weight matrices, not a model-quality claim.

The kernel wastes 25% of its code space. The two-bit encoding admits four codes $\{00, 01, 10, 11\}$; ternary weights use only three, leaving one code unused. An INT2 kernel using all four codes for $\{0, \pm1, \pm2\}$ or similar would not waste this capacity. This is a real inefficiency of the design choice. Its justification is theoretical rather than engineering: Cuevas Araneda [2026, Prop. 5] proves that the efficiency $\eta(k) = \log_2(k)/C_{\text{MAC}}(k)$ over symmetric weight alphabets is maximized at $k = 3$ ($\eta(3) = 52.8$ versus $\eta(5) = 38.7$, $\eta(7) = 31.2$). The design pays a 25% code-space waste to remain at the energy optimum.

What *is* genuinely ternary and contributes. First, the operation-count parity ($1.002\times$ lane-MACs versus fp16 analytical baseline, measured by the CPU simulator) is a correctness verification that the substrate executes a faithful transformer forward—this validation requires the actual ternary substrate as the measuring instrument, not the CUDA port. Second, the BitNet 2B-4T bit-exact reproduction is a substrate-level claim about alignment between data format and computation format that is operationally true for QAT-trained ternary models. Third, the substrate simulator itself (F0–F11, AVX2 host backend, `libter_k4.a` freestanding C ABI; §3) is the research vehicle for the project; the CUDA port is an acceleration of the inner matmul to make iteration tractable, not the production target. The follow-on work that would close the energy argument is gate-level or FPGA simulation of a balanced-ternary ALU compared against an equivalent-precision binary ALU, with J/MAC measured on real silicon. This is the next phase of the substrate program, and is outside the scope of the present paper.

6.4 What round 2 changes about the paper

The matmul-fabric measurement ($1.90\times$ over cuBLAS INT8 TC on Llama 1B at $M = 1$) is unchanged in magnitude; only the attribution changes (now ascribed to packed sub-byte storage density rather than to ternary arithmetic semantics, per §6.3). What round 2 *adds* is the end-to-end story: the previous draft argued the architectural win would survive engineering (with caveat). Round 2 measures the engineering and shows it does. The substrate runs both Llama 3.2 1B *and* BitNet 2B-4T end-to-end on RTX 3090 at production-grade throughput, with bit-exact reference match for BitNet [Microsoft, 2024].

7 What the simulator measures, and what it doesn't

The simulator measures **operation counts at the architectural level**, exact and substrate-independent:

- `TVMAC` count per forward (Llama 1B M=1: 45.86M).
- Lane-MAC count = `TVMAC` $\times$ 27 (1.238G).
- Memory bytes per layer / per forward (packed, INT8, Q8_0, fp16).
- Op-count breakdown (`TLOAD`, `TSTORE`, `TADD`, `TVADD`, `TVSUM`, `TJUMP`, ...).

These are the *what the hardware would actually do* numbers, substrate-invariant and exact.

The simulator does **not** directly measure:

- **Wall-clock time**: the CUDA numbers in §5 are wall-clock on a binary host. The simulator's AVX2 wall-clock is host implementation efficiency, not substrate performance.
- **Energy per operation (J/MAC)**: requires physical hardware measurement (FPGA prototype with ternary ALU versus binary ALU).
- **Silicon area per operation**: same; requires hardware fabrication.

The composition that the system-level energy argument needs is:

$$\text{energy}_{\text{total}} = \sum_{\text{op}} \text{count}(\text{op}) \cdot \text{energy\_per\_op}(\text{op}, \text{substrate}) \tag{4}$$

The simulator gives $\text{count}(\text{op})$ exactly. energy_per_op is an external input. With literature-typical numbers [Vudadha et al., 2016, Hayes et al., 2018] (balanced ternary CMOS $\sim 0.7\times$ area, $\sim 0.75\times$ energy per equivalent-precision MAC versus binary), the projected system-level energy advantage is real but unmeasured here.

8 Honest interpretation

8.1 What the paper defends

1. Modern transformer inference (Llama 1B, BitNet 2B-4T) is feasible on a balanced-ternary CPU substrate; both run end-to-end with finite outputs.
2. Op-count parity holds: $1.002\times$ lane-MACs versus the analytical fp16 baseline (measured by the CPU simulator on Llama 1B).
3. The ternary substrate stores models at 1.58 bits per weight ($8\times$ compression versus fp16, $5\times$ versus Q8_0) while preserving operation count.
4. Format A `tfloat` provides a precision/budget knob; 11 trits (1+5+5) preserves Llama 1B argmax with low logit-RMSE.
5. **Engineering result (Llama 1B end-to-end):** the CUDA port reaches 425 tokens/s on RTX 3090, $1.08\times$ faster than `llama.cpp` Q8_0, via four cumulative fixes (device-pointer scales, packed-trit warp-cooperative kernel, `cudaGraph` capture, rmsnorm/quant + silu/quant fusion). Underlying matmul fabric measurement at single-token gen: $1.90\times$ over cuBLAS INT8 TC, attributable per §6.3 to packed sub-byte memory density rather than ternary arithmetic semantics [Cuevas Araneda, 2026].
6. **Engineering result (BitNet 2B-4T end-to-end):** integration of `microsoft/BitNet` i2_s weights reaches 214 tokens/s with $\mathbf{8/8}$ BOS-greedy tokens bit-exact against the `llama-cli` reference. This validates substrate-data alignment for the QAT-trained ternary case, which is the regime where the alignment is semantically meaningful.
7. Kernel correctness on Llama 1B is validated three-way: scalar NumPy reference vs CUDA forward bit-exact $16/16$; reference architecture vs Q8_0 golden (without ternarization) $7/8$; ternarized Llama vs Q8_0 golden $0/16$ (confirming H1 prediction that post-training ternarization on non-QAT models breaks quality).

8.2 What the paper does not claim

1. **Throughput superiority of ternary arithmetic on binary silicon.** The $1.90\times$ matmul-fabric speedup is a sub-byte packing density result that a generic INT2 kernel feeding `__dp4a` could in principle match. Our ADD-only kernel, which exercises the truly ternary "no multiplication" property, matches but does not exceed the `__dp4a`-based packed kernel. The ternary-arithmetic energy advantage requires custom silicon to be observable; see §6.3.
2. **Production wins at prefill regimes.** At $M \geq 16$, cuBLAS tensor cores saturate and at $M \geq 64$ the hybrid dispatch margin is only $\sim 1.04\times$. The substrate's strongest regime is single-token gen.
3. **Quality preservation under post-training ternarization.** For Llama 1B, the $0/16$ golden-token match confirms that post-training quantization to ternary breaks non-QAT models. Substrate-data alignment is genuine only for the QAT-trained case (BitNet).
4. **Per-op energy advantage in deployed silicon.** The literature [Horowitz, 2014, Scherer et al., 2020, Cuevas Araneda, 2026] supports 3.8–$71.4\times$ ranges depending on process node, but we have not measured this on physical hardware. FPGA prototype is the next experiment.
5. **Novelty of the LUT-based / packed low-bit technique.** T-MAC [Wei et al., 2025], LUTMUL [Xie et al., 2025], and Platinum [Shan et al., 2025] are prior art for the principle that low-bit weights admit LUT-based or packed implementations that outperform conventional multiplication. Our contribution is the specific GPU packed-trit `__dp4a` kernel plus end-to-end engine integration plus bit-exact validation, not the principle itself.

9 Implications for silicon investment

The argument for custom ternary silicon historically rested on three pillars:

- **Throughput**: *partially* supported on commodity silicon. Packed sub-byte storage on consumer GPU (this work, Wei et al. [2025] on CPU, Xie et al. [2025] on FPGA) captures the bandwidth-bound portion of the win at single-token gen. As discussed in §6.3, this advantage is attributable to sub-byte packing density rather than to ternary arithmetic itself; an INT2 packing scheme would in principle obtain similar throughput.
- **Memory bandwidth**: empirically supported—$8\times$ compression versus fp16, $5\times$ versus Q8_0. This is shared with any sub-byte packed scheme; the ternary alphabet is one instantiation.
- **Per-op energy**: unmeasured here. The formal cost analysis of Cuevas Araneda [2026] (Prop. 2) gives a lower bound of $\rho_3 \geq 3.8\times$ in 45nm versus FP16; empirical measurements in 7nm [Ma et al., 2024] report up to $71.4\times$. This advantage is exclusively a property of the ternary ALU, not of the packed representation, and therefore cannot be captured by binary GPU regardless of kernel sophistication. Closing this argument requires FPGA or ASIC prototype measurement.

The throughput case for ternary substrates *is not made by this paper*—it is made, to the extent that it is made on commodity silicon, by sub-byte packed kernels for which ternary is one valid instantiation. The silicon-investment argument for custom ternary hardware therefore reduces to the energy case, which remains open.

The remaining specifically-silicon arguments are:

- **Per-op energy in a battery / edge regime**: if ternary CMOS gives even 25% energy/op savings, the integrated effect at scale (LLM inference is energy-dominated by memory access today) is meaningful.
- **Sub-byte memory bandwidth in extreme edge devices**: where DRAM bandwidth is the binding constraint, 1.58 bits/weight beats 8 bits by $5\times$ in available throughput.
- **Specialized inference accelerators for ternary-trained models**: the BitNet b1.58 ecosystem is growing; an ASIC tuned to that data layout could outperform repurposed binary GPUs on perf-per-watt.

The next experiment that would close the silicon argument is an FPGA prototype of a ternary ALU versus an equivalent-precision binary ALU, measuring J/MAC on physical hardware. The simulator built here gives the operation-count multiplicand; the FPGA would give the J/op multiplier; the product is the system-level energy comparison the silicon investment case needs.

10 Conclusion

We built a balanced-ternary CPU simulator capable of running Llama 3.2 1B and BitNet b1.58 2B-4T end-to-end, validated three architectural hypotheses (Format B preserves Llama quality, Format A trit-budget tradeoff, BitNet substrate-data alignment), and ported the matmul fabric to CUDA. The two engineering results this paper defends are: (a) end-to-end Llama 3.2 1B inference at $\mathbf{425}$ tokens/s on RTX 3090, $\mathbf{1.08}\times$ faster than `llama.cpp` Q8_0, via the round-2 fixes in §6; and (b) bit-exact reproduction of `microsoft/BitNet` 2B-4T at $\mathbf{214}$ tokens/s with $\mathbf{8/8}$ BOS-greedy tokens matching the reference. The matmul-fabric measurement that underlies (a)—$1.90\times$ over cuBLAS INT8 TC at single-token gen—is attributable to packed sub-byte storage density rather than to ternary arithmetic per se, following the formal decomposition of Cuevas Araneda [2026] and confirmed by our own ADD-only experiment. We adopt that decomposition explicitly in §6.3.

The throughput case for ternary substrates on commodity binary silicon is, on this evidence, equivalent to the case for any sub-byte packed representation that exploits memory bandwidth (1.58 bits per weight fits the same kernel template that INT2 or any other two-bit scheme would fit). The case that remains specifically ternary is energetic: balanced-ternary CMOS circuits may save 25–71% J/MAC relative to equivalent-precision binary [Horowitz, 2014, Scherer et al., 2020, Cuevas Araneda, 2026], but this paper does not measure it. The substrate simulator (F0–F11, AVX2 backend, `libter_k4.a` freestanding C ABI) was designed as the research vehicle for exactly that question; the CUDA port was an iteration aid. The next phase of the project is gate-level or FPGA prototyping of a balanced-ternary ALU versus a binary ALU, which would supply the J/op multiplier that, combined with the operation-count multiplicand the simulator already gives, closes the silicon-investment argument. We thank Felipe Cuevas Araneda [Cuevas Araneda, 2026] for the formal analysis and critique that informed this revision.

Reproduction

All artifacts and tests are in the ter repository at https://github.com/xaskasdf/ter. Headline experiments:

- Llama 1B Q8_0 forward: tests/test_llama_smoke.cpp
- BitNet 2B-4T forward (real GGUF): tests/test_bitnet_forward.cpp
- Format A trit-budget sweep on Llama 1B: tests/test_format_a_vs_b_llama.cpp
- BitNet quality on Llama 1B (post-training): tests/test_llama_bitnet_quality.cpp
- Best CUDA packed kernel: cuda/ter_cuda_packed_v6.cu
- End-to-end CUDA forward (Llama, packed weights, 425 t/s): cuda/ter_cuda_forward_packed.cu
- End-to-end CUDA forward (BitNet 2B-4T real weights, 214 t/s, 8/8 bit-exact): cuda/ter_cuda_forward_bitnet.cu
- BitNet i2_s GGUF → packed converter: tools/convert_bitnet_to_packed.py
- Hybrid per-shape kernel dispatcher (M=1, 16, 64): cuda/ter_cuda_hybrid_dispatch.cu
- INT4 TC via wmma::s4 fragments: cuda/ter_cuda_int4_tc.cu

Build (Mac AVX2):

```
cmake -S . -B build && cmake --build build
ctest --test-dir build --output-on-failure
```

Build CUDA bench (NVIDIA GPU + CUDA toolkit, ≥ Ampere SM 8.6):

```
nvcc -O3 -arch=sm_86 -std=c++17 cuda/ter_cuda_packed_v6.cu \
     -lcublas -o ter_cuda_packed_v6
./ter_cuda_packed_v6 100
```

Acknowledgments

The simulator borrows GGUF loading and tokenizer infrastructure from the author's prior ntransformer C++/CUDA Llama inference engine [Cortes, 2026b]. BitNet b1.58 architecture follows [Ma et al., 2024] and the microsoft/BitNet implementation [Microsoft, 2024]. llama.cpp [Gerganov, 2023] provided the production fp16 baseline measurements on the same RTX 3090. The CUDA __dp4a intrinsic is the core acceleration primitive enabling the 1.90× win [NVIDIA, 2024b, 2020].

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *EMNLP*, 2023.

N. P. Brusentsov. Setun: The first balanced ternary computer, 1959. Moscow State University, 1959. Historical predecessor of balanced-ternary computing architectures.

Samuel Cortes. Brandon-tiny 10m: A 3-phase training pipeline for ultra-small instruction-following language models. 2026a.

Samuel Cortes. ntransformer: A c++/cuda inference engine for Llama 3.2 1B. 2026b.

Felipe Cuevas Araneda. Análisis de eficiencia energética en multiplicación de matrices con pesos discretizados: del régimen ternario a la extensión quinaria vía table lookup. 2026. Formal energy-cost analysis with $k = 3$ optimality proof and MAC-to-LUT reduction; includes a critical review of this work that motivated the revisions made in this version of the paper.

Georgi Gerganov. llama.cpp: Inference of llama model in pure c/c++. https://github.com/ggerganov/llama.cpp, 2023. Accessed 2026-05-10.

Georgi Gerganov. GGML quantization formats and GGUF file structure. 2024.

John P. Hayes et al. Energy-efficient CMOS implementation of balanced ternary adders and multipliers. *IEEE Transactions on Circuits and Systems I*, 2018. Representative literature reference; multiple sources report 25–40% energy savings per equivalent-precision MAC for balanced ternary CMOS designs.

Mark Horowitz. 1.1 computing's energy problem (and what we can do about it). In *IEEE International Solid-State Circuits Conference (ISSCC)*, pages 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323.

Fengfu Li, Bin Liu, Xiaoxing Wang, Bo Zhang, and Junchi Yan. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016.

Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

Lukas Meiner, Jens Mehnert, and Alexandru Paul Condurache. PROM: Prioritize reduction of multiplications over lower bit-widths for efficient CNNs. *arXiv preprint arXiv:2505.03254*, 2025.

Meta AI. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/, 2024.

Microsoft. BitNet: Official inference framework for 1-bit llms. https://github.com/microsoft/BitNet, 2024. Accessed 2026-05-10.

NVIDIA. NVIDIA Ampere Architecture Whitepaper. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf, 2020.

NVIDIA. cuBLAS: NVIDIA GPU-accelerated basic linear algebra subroutines. https://docs.nvidia.com/cuda/cublas/, 2024a.

NVIDIA. CUDA C++ programming guide: Simd video instructions. https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2024b. Accessed 2026-05-10.

Ye Qiao, Zhiheng Chen, Yifan Zhang, Yian Wang, and Sitao Huang. TeLLMe: An energy-efficient ternary LLM accelerator for prefilling and decoding on edge FPGAs. *arXiv preprint arXiv:2504.16266*, 2025.

Moritz Scherer, Georg Rutishauser, Lukas Cavigelli, and Luca Benini. CUTIE: Beyond PetaOp/s/W ternary DNN inference acceleration with better-than-binary energy efficiency. *arXiv preprint arXiv:2011.01713*, 2020.

Haoxuan Shan, Cong Guo, Chiyue Wei, Feng Cheng, Junyao Zhang, Hai Helen Li, and Yiran Chen. Platinum: Path-adaptable LUT-based accelerator tailored for low-bit weight matrix multiplication. *arXiv preprint arXiv:2511.21910*, 2025.

Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.

Chetan Vudadha, A. Surya, Sruthi Agrawal, and M. B. Srinivas. A survey on ternary logic gate designs in CMOS technology. *IEEE Access*, 4:7530–7550, 2016.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge. In *European Conference on Computer Systems (EuroSys)*, 2025. doi: 10.1145/3689031.3696099.

Yanyue Xie, Zhengang Li, Dana Diaconu, Suranga Handagala, Miriam Leeser, and Xue Lin. LUTMUL: Exceed conventional FPGA roofline limit by LUT-based efficient multiplication for neural network inference. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2025. doi: 10.1145/3658617.3697687.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *NeurIPS*, 2019.